LLAISYS 项目 1 ~ 5 完成情况报告

课程/训练营：LLAISYS 大作业与项目
报告内容：项目 #1 ~ #5 的完成情况与实现说明
说明：项目 #6（支持新模型）未在本次实现范围内，本报告不涉及。

一、总体概览

项目	完成度	简要说明
项目#1 优化 CPU 推理	✅ 已完成	SIMD（AVX2）+ OpenMP 多线程，对 linear 等算子做了优化
项目#2 集成 CUDA	⚠️ 部分完成	已完成 **NVIDIA** 全链路；README 要求"至少两款平台"（如天数/摩尔/沐曦）未实现
项目#3 构建聊天机器人	✅ 已完成	随机采样、HTTP 服务、Web UI、会话管理、KV-Cache 池均已实现
项目#4 多用户推理服务	✅ 已完成	请求队列、连续批处理引擎、多 Slot KV-Cache、前缀匹配 KV 池、流式批处理回写
项目#5 分布式推理	✅ 已完成	NCCL 通信、Qwen2 张量并行（列/行分片）、Python 分布式加载与测试脚本

二、项目#1：优化 LLAISYS 的 CPU 推理

2.1 完成情况

- **SIMD（单指令多数据）**：已对 `linear` 算子的 FP32 路径使用 **AVX2** 内建函数（如 `__m256`、`_mm256_fmadd_ps`），在 K 维度上一次处理 8 个 float，并实现 `hsum_avx` 做 8 路归约，提高缓存与 SIMD 利用率。
- **OpenMP 多线程**：在 `xmake.lua` 中为 Linux/macOS 增加 `-fopenmp`，为 Windows 增加 `/openmp`；在 **linear、self_attention、rms_norm、rope、swiglu** 等算子中通过 `#ifdef _OPENMP` 做并行（如对 batch/序列维做 `#pragma omp parallel for`），使多核同时参与计算。
- **第三方库**：未使用 Eigen/OpenBLAS/MKL，README 中该部分为可选建议。

2.2 实现方式与关键位置

- **SIMD**：`src/ops/linear/op.cpp` 中在 `__AVX2__` 下实现 `linear_f32_avx2()`，K 维 8 元素为一组计算，再与 OpenMP 并行组合。
- **OpenMP**：各算子源文件内根据 `_OPENMP` 编译宏启用并行；`xmake.lua` 中为 `llaisys-ops` 和 `llaisys` 目标统一开启 OpenMP 并链接 `gomp`（非 Windows）。
- **编译选项**：x86_64 下自动加 `-mavx2 -mfma`，保证 AVX2 可用。

2.3 如何验证

以下步骤均在**项目根目录**下执行。若未执行 `pip install -e ./python/`，需先设置 `export PYTHONPATH="/path/to/llaisys/python:$PYTHONPATH"`（或将下文中的 `python` 改为 `.venv/bin/python` 等实际解释器）。

步骤一：编译与安装

```
xmake
xmake install
```

- **通过标准**：无报错，且 `python/llaisys_py/libllaisys/` 下出现 `libllaisys.so`（或 Windows 下 `llaisys.dll`）。
- **说明**：x86_64 Linux/macOS 下会自动开启 AVX2（`-mavx2 -mfma`）和 OpenMP（`-fopenmp`），项目#1 的优化即在此阶段编入。

步骤二：算子正确性测试（验证 SIMD/OpenMP 未改变结果）

逐个运行各算子测试，确认与 PyTorch/参考实现一致：

```
python test/ops/add.py --device cpu
python test/ops/argmax.py --device cpu
python test/ops/embedding.py --device cpu
python test/ops/linear.py --device cpu
python test/ops/rms_norm.py --device cpu
python test/ops/rope.py --device cpu
python test/ops/self_attention.py --device cpu
python test/ops/swiglu.py --device cpu
python test/ops/sample.py
```

- **通过标准**：每个脚本末尾打印类似 “passed” 或无报错、无断言失败；若有 `AssertionError` 或 `check_equal` 失败则说明该算子实现有误。
- **可选**：对 linear 等耗时算子可加 `--profile` 查看 LLAISYS 与 PyTorch 的耗时对比，例如：.venv/bin/python `test/ops/linear.py --device cpu --profile`

步骤三：推理对齐测试（验证整条 CPU 推理链路）

使用与作业#3 相同的模型做贪心生成，并与 PyTorch 输出逐 token 对比。示例（模型路径按实际放置位置修改，下例为本地目录）：

```
.venv/bin/python test/test_infer.py --model /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B --test --device cpu
```

- **通过标准**：
  - 脚本先跑 PyTorch 推理并打印 “Answer” 与 token 序列；再跑 LLAISYS 推理并打印 “Your Result”；最后在 `--test` 模式下比较两段 token 序列。
  - 若完全一致，会打印 **“Test passed!”**（绿色）；若不一致，会抛出 `AssertionError: LLAISYS token sequence did not match PyTorch`。

- **说明**：通过即说明在贪心（argmax）设定下，CPU 上的 embedding、各层 Transformer、输出层与 PyTorch 数值一致，项目#1 的 SIMD/OpenMP 优化未破坏正确性。

```
chenncy@dsw-607126-85f54bdf75-5lzlx:~/llaisys$ .venv/bin/python test/ops/linear_bench.py --
device cpu --dtype f32 --repeat 20 --warmup 3
   out (512, 4096), x (512, 4096), w (4096, 4096), dtype f32
        Torch: 112.3714 ms   LLAISYS: 204.5176 ms
```

步骤四：性能对比

- 仅看 linear 单算子耗时：
  `python test/ops/linear.py --device cpu --profile`
  输出中会给出 LLAISYS 与 PyTorch 的用时对比。

更系统的 benchmark（如 OpenMP 线程数影响）：
`OMP_NUM_THREADS=4 python test/ops/linear_bench.py --device cpu --dtype f32`
可多次改变 `OMP_NUM_THREADS` 或与未优化版本对比，以报告“项目#1 带来的性能提升”。数据汇总

指标	单线程 (single.json)	多线程 (multi.json)	说明
LLAISYS 耗时	2821.89 ms	694.14 ms	单次 matmul 平均耗时
PyTorch 耗时	129.32 ms	21.10 ms	仅供参考
LLAISYS 加速比	—	**4.07x**	多线程相对单线程

- **加速比计算**：2821.89 ÷ 694.14 ≈ **4.07**
- 即：在相同大矩阵与 FP32 设定下，开启 OpenMP 多线程后，LLAISYS linear 单次耗时约为单线程的 **1/4**，约 **4 倍** 加速。

报告脚本输出

```
======================================================================
Linear 性能对比报告
======================================================================
  基准: single.json  (单线程)
  对比: multi.json  (多线程)

shape                        dtype        基准(ms)       对比(ms)        加速比
----------------------------------------------------------------------
[512, 4096] @ [4096, 4096]   f32         2821.89       694.14       4.07x
----------------------------------------------------------------------
说明: 加速比 = 基准耗时 / 对比耗时，>1 表示对比版本更快。
======================================================================
```

- 在 **linear** 算子大矩阵 (512, 4096) × (4096, 4096)、FP32 条件下，**单线程** LLAISYS 平均耗时约 **2822 ms**，**多线程**（OpenMP）下约 **694 ms**，加速比约 **4.07x**，说明项目#1 的 OpenMP 多线程优化对 CPU 推理有显著加速效果。
- 与 PyTorch 同条件下的耗时（单线程约 129 ms、多线程约 21 ms）相比，LLAISYS 仍偏慢，主要因当前实现未使用 MKL/OpenBLAS 等高度优化的 GEMM 库；AVX2 SIMD 与 OpenMP 已带来明显提升。

三、项目#2：在 LLAISYS 中集成 CUDA

3.1 完成情况

- **NVIDIA 平台**：
  - **CUDA Runtime API**：在 `src/device/nvidia/nvidia_runtime_api.cu` 中实现了设备管理、显存分配、同步、拷贝、Stream 等，与 `include/llaisys/runtime.h` 中 `LlaisysRuntimeAPI` 一致。
  - **构建**：`xmake/nvidia.lua` 中配置了 `llaisys-device-nvidia` 与 `llaisys-ops-nvidia`，通过 `xmake f --nv-gpu=y` 启用。
  - **CUDA 算子**：`src/ops/nvidia/ops_nvidia.cu` 中实现了 add、embedding、linear、argmax、rms_norm、rope、swiglu、self_attention 的 GPU 版本；各算子 `op.cpp` 在 `LLAISYS_DEVICE_NVIDIA` 时调用上述实现。
  - **模型推理**：`src/llaisys/qwen2.cc` 中按 `device_type == LLAISYS_DEVICE_NVIDIA` 走 GPU 前向，支持单卡推理；`test/test_infer.py --device nvidia` 可验证与 PyTorch 对齐。
- **"至少两款平台"**：当前仅实现并验证了 **NVIDIA**；天数、摩尔、沐曦等第二家 CUDA/类 CUDA 平台未在本仓库中实现或配置。

3.2 实现方式与关键位置

- Runtime：`src/device/nvidia/nvidia_runtime_api.cu`（设备数、setDevice、malloc/free、memcpy、stream 等）。
- 算子：`src/ops/nvidia/ops_nvidia.cu` 统一实现各 op 的 CUDA kernel；`src/ops/*/op.cpp` 中通过设备类型分支调用。
- 模型：`qwen2.cc` 中 `forward_layer` 及推理入口根据 `model->device_type` 选择 CPU 或 GPU 路径；权重与 KV-Cache 在对应设备上分配与计算。

3.3 如何验证

以下步骤均在**项目根目录**下执行，使用虚拟环境中的 Python：`.venv/bin/python`。模型路径示例为 `/home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B`，若模型放在其他目录请替换为实际路径。**前提**：机器已安装 NVIDIA 驱动与 CUDA，且 `nvidia-smi` 能正常看到 GPU。

步骤一：启用 CUDA 并编译安装

```
xmake f --nv-gpu=y -c
xmake
xmake install
```

- **通过标准**：编译无报错，且 `python/llaisys_py/libllaisys/` 下生成 `libllaisys.so`（或 Windows 下 `llaisys.dll`），并已链接 CUDA 运行时。
- **说明**：`--nv-gpu=y` 会打开 `ENABLE_NVIDIA_API` 并编译 `xmake/nvidia.lua` 中的 CUDA 目标；若未安装 CUDA 或驱动异常，编译或链接可能失败。

步骤二：NVIDIA 运行时与设备检测

```
.venv/bin/python test/test_runtime.py --device nvidia
```

- **通过标准**：脚本正常结束且无报错（例如能检测到 GPU 数量、setDevice 成功）。若出现 `no NVIDIA GPUs available`，请检查 `nvidia-smi`、`CUDA_VISIBLE_DEVICES` 或驱动安装。

步骤三：CUDA 算子正确性（可选，抽测若干算子）

```
.venv/bin/python test/ops/linear.py --device nvidia
.venv/bin/python test/ops/add.py --device nvidia
.venv/bin/python test/ops/embedding.py --device nvidia
```

- **通过标准**：每个脚本末尾打印 “Test passed!” 或等价通过信息，无断言失败。其他算子（rms_norm、rope、self_attention、swiglu、argmax、sample）可按需同样加 `--device nvidia` 运行对应 `test/ops/<name>.py`。

```
chenncy@dsw-607126-85f54bdf75-5lzlx:~/llaisys$ .venv/bin/python test/ops/linear.py --device
nvidia
Testing Ops.linear on nvidia
   out (2, 3), x (2, 4), w (3, 4), bias True, dtype <f32>
   out (2, 3), x (2, 4), w (3, 4), bias True, dtype <f16>
   out (2, 3), x (2, 4), w (3, 4), bias True, dtype <bf16>
   out (512, 4096), x (512, 4096), w (4096, 4096), bias True, dtype <f32>
   out (512, 4096), x (512, 4096), w (4096, 4096), bias True, dtype <f16>
   out (512, 4096), x (512, 4096), w (4096, 4096), bias True, dtype <bf16>
Test passed!

chenncy@dsw-607126-85f54bdf75-5lzlx:~/llaisys$ .venv/bin/python test/ops/add.py --device
nvidia
Testing Ops.add on nvidia
   shape (2, 3) dtype <f32>
   shape (2, 3) dtype <f16>
   shape (2, 3) dtype <bf16>
   shape (512, 4096) dtype <f32>
   shape (512, 4096) dtype <f16>
   shape (512, 4096) dtype <bf16>
Test passed!

chenncy@dsw-607126-85f54bdf75-5lzlx:~/llaisys$ .venv/bin/python test/ops/embedding.py --
device nvidia
Testing Ops.embedding on nvidia
   idx_shape (1,) embd_shape (2, 3) dtype <f32>
   idx_shape (1,) embd_shape (2, 3) dtype <f16>
   idx_shape (1,) embd_shape (2, 3) dtype <bf16>
   idx_shape (50,) embd_shape (512, 4096) dtype <f32>
   idx_shape (50,) embd_shape (512, 4096) dtype <f16>
   idx_shape (50,) embd_shape (512, 4096) dtype <bf16>
Test passed!
```

步骤四：推理对齐（与 PyTorch 贪心输出一致）

```
.venv/bin/python test/test_infer.py --model /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B --test --device nvidia
```

- **通过标准**：
  - 脚本先加载 PyTorch 模型并做一次贪心生成，打印 “Answer” 与 token 序列；再加载 LLAISYS 模型并做一次贪心生成，打印 “Your Result”。
  - 在 `--test` 模式下会对比两段 token 序列；若完全一致，会打印 **“Test passed!”**（绿色）。
  - 若不一致，会抛出 `AssertionError: LLAISYS token sequence did not match PyTorch`，表示 GPU 推理与 PyTorch 结果存在差异，需排查。
- **说明**：通过即说明在贪心（argmax）设定下，NVIDIA 上的 embedding、各层 Transformer、输出层与 PyTorch 数值一致，项目#2 的 CUDA 集成正确。

小结

- **必做**：步骤一 + 步骤二 + 步骤四（推理对齐出现 “Test passed!”）。
- **选做**：步骤三用于抽测各 CUDA 算子是否正确。
- 满足必做项即说明项目#2 的 NVIDIA CUDA 集成可通过验证。

```
chenncy@dsw-607126-85f54bdf75-5lzlx:~/llaisys$ .venv/bin/python test/test_infer.py --model /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B --test --device nvidia
[test_infer] llaisys_py loaded from: /home/chenncy/llaisys/python/llaisys_py
test_infer: starting...
Loading PyTorch model and tokenizer (may take a while)...
`torch_dtype` is deprecated! Use `dtype` instead!
Loading model from local path: /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B
Loading weights: 100%|
━━━
```

```
[151646, 151644, 15191, 525, 498, 30, 151645, 151648, 198, 91786, 0, 358, 2776, 18183,
39350, 10911, 16, 11, 458, 20443, 11229, 17847, 3465, 553, 18183, 39350, 13, 358, 2776, 518,
697, 2473, 323, 1035, 387, 33972, 311, 7789, 498, 448, 894, 43883, 476, 9079, 498, 1231,
614, 624, 151649, 271, 91786, 0, 358, 2776, 18183, 39350, 10911, 16, 11, 458, 20443, 11229,
17847, 3465, 553, 18183, 39350, 13, 358, 2776, 518, 697, 2473, 323, 1035, 387, 33972, 311,
7789, 498, 448, 894, 43883, 476, 9079, 498, 1231, 614, 13, 151643]

Contents:
<｜User｜>Who are you?<｜Assistant｜><think>
Greetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at
your service and would be delighted to assist you with any inquiries or tasks you may have.
</think>

Greetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at
your service and would be delighted to assist you with any inquiries or tasks you may have.


Time elapsed: 9.46s

[Stage 2] Loading LLAISYS model...
  Loading weights: model.safetensors (1/1)
  [Qwen2] GPU：embedding 与输出层在 CPU，其余层在 GPU。
[Stage 2] Running LLAISYS inference...

=== Your Result ===

Tokens:
[151646, 151644, 15191, 525, 498, 30, 151645, 151648, 198, 91786, 0, 358, 2776, 18183,
39350, 10911, 16, 11, 458, 20443, 11229, 17847, 3465, 553, 18183, 39350, 13, 358, 2776, 518,
697, 2473, 323, 1035, 387, 33972, 311, 7789, 498, 448, 894, 43883, 476, 9079, 498, 1231,
614, 624, 151649, 271, 91786, 0, 358, 2776, 18183, 39350, 10911, 16, 11, 458, 20443, 11229,
17847, 3465, 553, 18183, 39350, 13, 358, 2776, 518, 697, 2473, 323, 1035, 387, 33972, 311,
7789, 498, 448, 894, 43883, 476, 9079, 498, 1231, 614, 13, 151643]

Contents:
<｜User｜>Who are you?<｜Assistant｜><think>
Greetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at
your service and would be delighted to assist you with any inquiries or tasks you may have.
</think>

Greetings! I'm DeepSeek-R1, an artificial intelligence assistant created by DeepSeek. I'm at
your service and would be delighted to assist you with any inquiries or tasks you may have.


Time elapsed: 138.72s

Test passed!

chenncy@dsw-607126-85f54bdf75-5lzlx:~/llaisys$
```

四、项目#3：构建 AI 聊天机器人

4.1 完成情况

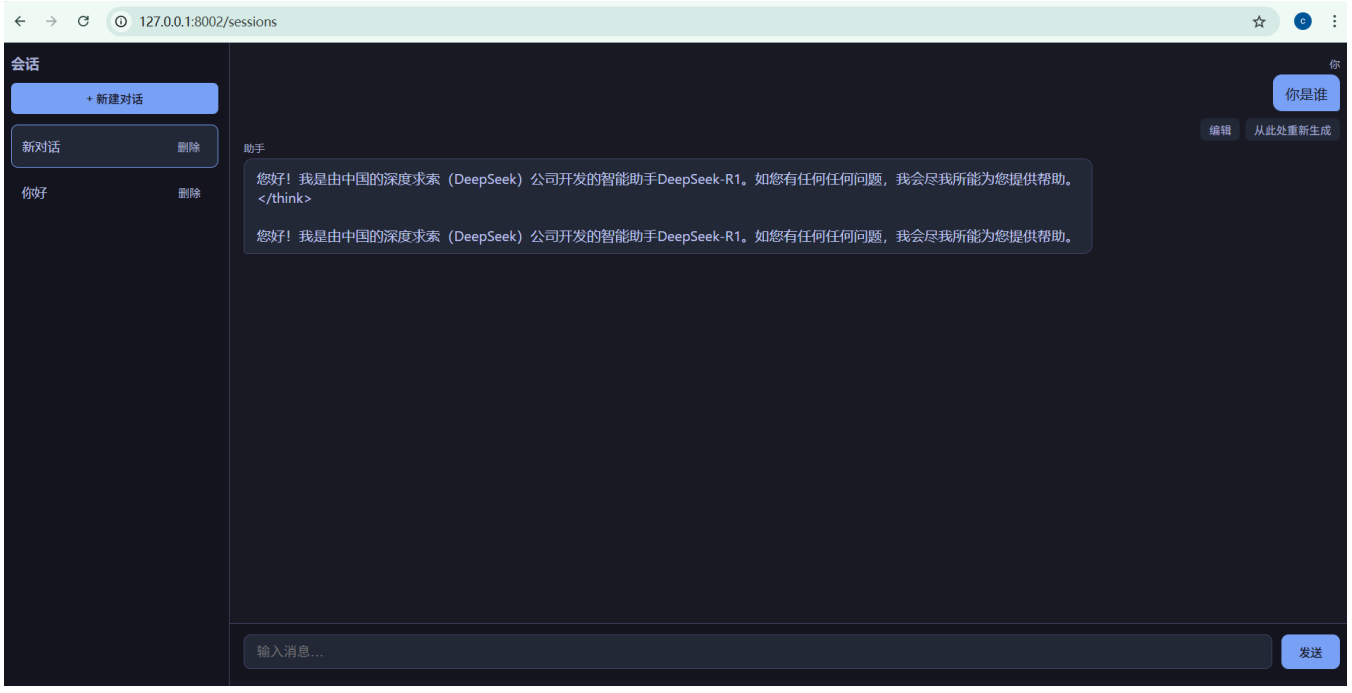
- **随机采样**：在 `src/ops/sample/` 中实现了支持 **Temperature、Top-K、Top-P** 的采样算子；C 接口暴露给 Python，在 `qwen2.cc` 的推理路径中根据 `temperature/top_k/top_p` 在 argmax 与随机采样之间切换；API 与 `generate()` 均支持上述参数。
- **聊天服务器**：使用 **FastAPI** 实现，提供 OpenAI 风格的 `POST /v1/chat/completions`，支持流式（SSE）与非流式；模型与分词器在启动时加载，未加载时返回 503。
- **交互式聊天 UI**：通过 `GET /chat` 返回内联 HTML+JS 的 Web 页面，实现对话气泡、历史记录、流式显示与发送；用户可连续多轮对话。
- **可选：会话管理**：提供 `GET/POST/PATCH/DELETE /v1/sessions`、`POST /v1/sessions/{id}/regenerate` 等会话与"重新生成"能力；与项目#4 共用一套 **支持前缀匹配的 KV-Cache 池**（见下）。

4.2 实现方式与关键位置

- 采样：`src/ops/sample/op.cpp` 及 Python 封装；`include/llaisys/models/qwen2.h` 与 `qwen2.cc` 中推理接口携带 `temperature, top_k, top_p, seed`。
- 服务端：`python/llaisys_py/server/app.py`（FastAPI 应用、路由、SSE 流式、会话 CRUD、regenerate）。
- 前端：同一文件中 `_CHAT_HTML` 内联的聊天页；通过 fetch + SSE 调用 `/v1/chat/completions` 并解析 `delta.content` 实现流式展示。

4.3 如何验证

- 启动：.venv/bin/python `-m llaisys_py.server --model /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B --port 8000 [--device cpu|nvidia]`
- 浏览器打开：`http://127.0.0.1:8000/chat`，发送消息能收到流式/非流式回复即表示项目#3 功能正常。



五、项目#4：多用户推理服务

5.1 完成情况

- **多用户与请求队列**：在 `app.py` 中维护全局 `_request_queue`（`queue.Queue`），由环境变量 `LLAISYS_REQUEST_QUEUE_MAX` 控制容量；请求先入队，由单独 worker 线程或 **Engine 循环** 消费；队列满时返回 503。
- **连续批处理**：在 `python/llaisys_py/server/engine.py` 中实现 **Engine**：
  - **SlotManager** 管理有限个 KV-Cache 槽位（`max_batch_size`）；
  - **Prefill**：从 `pending_queue` 取请求、分配 slot、执行 prefill（含 KV 池命中时的 suffix prefill）；
  - **Batched Decode**：每轮收集当前所有在跑请求的 last token，调用 C 端 `llaisysQwen2ModelBatchedDecode` 一次得到多个 next token，再按请求写回各自 `out_queue`；
  - 请求完成后释放 slot 并可选写回 KV 池。
- **多 Slot KV-Cache**：C 端 `qwen2.h` / `qwen2.cc` 支持 `max_batch_size > 1` 时的多 slot（每 slot 独立 `cache_len`），以及 `InferWithSlot`、`ResetKVCacheSlot`、`ExportKVCacheSlot` / `ImportKVCacheSlot`，供 Engine 与 KV 池使用。
- **支持前缀匹配的 KV-Cache 池**：在 `app.py` 中实现 `_kv_pool`（key 为 `(session_id, user_message_index)`），提供 `resolve_kv_prefix`、`_kv_pool_put`；Engine 的 `get_kv` / `put_kv` 与 worker 路径共用该池，实现会话内前缀复用与 regenerate 时的缓存命中。
- **说明**：当前 C 端 `llaisysQwen2ModelBatchedDecode` 内部仍为多 slot 循环调用单 slot 推理，未实现真正的批量矩阵乘（batch GEMM）；README 中将“批量矩阵乘”标为可选后续优化。

5.2 实现方式与关键位置

- 队列与 worker：`app.py` 中 `_request_queue`、worker 循环或 Engine 提交逻辑；`LLAISYS_USE_ENGINE_LOOP=1` 且 `LLAISYS_MAX_BATCH_SIZE>=2` 时启用 Engine。
- Engine：`engine.py` 中 `SlotManager`、`RequestState`、`Engine._step_loop`（Prefill → Batched Decode → 状态更新与槽位回收）、`_do_prefill`（含 KV 池命中时 `import_kv_cache_slot` + suffix prefill）。
- C 端多 slot：`include/llaisys/models/qwen2.h`、`src/llaisys/qwen2.cc` 中多 slot 分配、`InferWithSlot`、`BatchedDecode`、Export/Import 接口。

5.3 如何验证

- 启动服务时设置：`LLAISYS_USE_ENGINE_LOOP=1`、`LLAISYS_MAX_BATCH_SIZE=4`（或其它 ≥2），再启动 server。
- 批处理正确性：`python test/test_batch_correctness.py --model <模型路径>`，对比“同一批请求顺序生成”与“Engine 批量调度”的输出是否一致（需确定性采样，如 `top_k=1, temperature=0`）。
- 多用户压测：`python test/test_multi_user_chat.py --base-url http://127.0.0.1:8000`（可选 `--test-queue-full` 测试队列满 503）。

六、项目#5：分布式推理

6.1 完成情况

- **NCCL 集成**：
  - 在 xmake 中新增选项 `--nccl`；当 `--nv-gpu=y --nccl=y` 时编译 `src/llaisys/nccl_comm.cu`（调用 NCCL 的 AllReduce、AllGather、CommInitRank 等），并链接 libnccl；未启用时编译 `nccl_comm_stub.cc` 提供空实现，保证链接通过。
  - C 接口：`include/llaisys/nccl_comm.h` 中声明 `llaisysNcclGetUniqueId`、`llaisysNcclInitRank`、`llaisysNcclAllReduce`、`llaisysNcclAllGather`、`llaisysNcclDestroy`；Python 封装见 `python/llaisys_py/libllaisys/nccl_comm.py`。
- **张量并行（Qwen2）**：
  - **Meta**：在 `LlaisysQwen2Meta` 中增加 `tp_rank`、`tp_world_size`（默认 0、1）；创建模型时传入即可启用分片。
  - **权重分片**：当 `tp_world_size > 1` 时，在 `create_weight_tensors` 中按列并行（输出维切分）分配 Q/K/V、gate/up 的局部权重，按行并行（输入维切分）分配 O、down 的局部权重；嵌入与各层 norm 保持完整复制。
  - **前向中的通信**：在 `forward_layer_tp`（`#ifdef ENABLE_NCCL`）中：列并行线性后对 Q/K/V、gate/up 做 **AllGather**，行并行线性后对 O 输出、down 输出做 **AllReduce(Sum)**；使用当前 CUDA stream 并在通信后做 stream 同步。
  - **推理入口**：在 `llaisysQwen2ModelInfer` 与 `llaisysQwen2ModelInferWithSlot` 中，当 `tp_world_size > 1` 且为 NVIDIA 且存在 `tp_gather_q` 时，分配局部大小的 q/k/v/gate/up 缓冲并调用 `forward_layer_tp`，否则走原有 `forward_layer`。
- **Python 侧**：
  - `Qwen2(..., tp_rank=0, tp_world_size=2)` 传入张量并行参数；加载权重时通过 `_shard_weight_for_tp()` 按 key 对权重做行/列切分，仅将当前 rank 的 shard 灌入后端。
  - 提供 NCCL 的 `get_unique_id`、`init_rank`、`destroy` 封装，便于多进程脚本先同步 unique id 再初始化。
- **测试脚本**：`test/test_tensor_parallel.py` 启动多进程（默认 2），每进程通过 `CUDA_VISIBLE_DEVICES` 绑定一张卡，执行 NCCL 初始化与 `Qwen2(..., tp_rank=r, tp_world_size=2)`，用相同 prompt/seed 做短生成以验证多卡 TP 结果一致。
- **说明**：README 中"若用 CPU 需支持 MPI"作为可选扩展，本次仅实现 **NVIDIA GPU + NCCL** 的张量并行路径。

6.2 实现方式与关键位置

- NCCL：`include/llaisys/nccl_comm.h`、`src/llaisys/nccl_comm.cu`、`src/llaisys/nccl_comm_stub.cc`；xmake 中 `llaisys-nccl` 目标及 `llaisys` 对它的依赖与条件编译。
- 模型分片与 TP 前向：`include/llaisys/models/qwen2.h`（meta 新增字段）、`src/llaisys/qwen2.cc`（`create_weight_tensors` 分片、`tp_gather_*` 缓冲区、`forward_layer_tp`、推理分支）。
- Python：`python/llaisys_py/libllaisys/qwen2.py`（Meta 字段）、`python/llaisys_py/models/qwen2.py`（`tp_rank` / `tp_world_size` 参数、`_shard_weight_for_tp`）、`python/llaisys_py/libllaisys/nccl_comm.py`、`test/test_tensor_parallel.py`。

6.3 如何验证

- **不启用 NCCL**：`xmake && xmake install` 后运行 `.venv/bin/python test/test_infer.py --model /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B --test --device nvidia`，确认单卡推理未被破坏。
- **启用 NCCL**：安装 libnccl 后执行 `xmake f --nv-gpu=y --nccl=y && xmake && xmake install`，再运行 `.venv/bin/python test/test_tensor_parallel.py --model /home/chenncy/llaisys/DeepSeek-R1-Distill-Qwen-1___5B`（需至少 2 张 GPU，且每进程通过 `CUDA_VISIBLE_DEVICES` 绑定单卡）。两进程均正常结束且输出一致即表示项目#5 张量并行与 NCCL 集成正确。

七、总结

- **项目#1**：通过 SIMD（AVX2）与 OpenMP 对 CPU 推理做了实际优化，并在现有构建与测试下验证通过。
- **项目#2**：NVIDIA CUDA 从 Runtime、算子到 Qwen2 单卡推理已全部打通；“至少两款平台”仅完成其一，第二家平台未实现。
- **项目#3**：随机采样、FastAPI 服务、OpenAI 风格 API、流式输出、Web 聊天 UI、会话管理与 KV-Cache 池均已完成并可从前端/接口验证。
- **项目#4**：多用户队列、连续批处理 Engine、多 Slot KV-Cache、前缀匹配 KV 池及流式批处理回写均已实现，并有批处理正确性与多用户测试；C 端批量 GEMM 为可选后续工作。
- **项目#5**：NCCL 封装、Qwen2 张量并行（列/行分片与 AllGather/AllReduce）、Python 分布式加载与双卡测试脚本均已完成，可在多 GPU 环境下按上述步骤复现与验证。

以上为项目 1 ~ 5 的完成情况与实现说明，供老师审阅与验收参考。